```
# Imports
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
```

```
SEED = 42
np.random.seed(SEED)
```

```
# Part (a): Data Pipeline
X, y = load_breast_cancer(return_X_y=True)
y = 1 - y
y = y.reshape(-1, 1)
```

```
X_train, X_tmp, y_train, y_tmp = train_test_split(
    X, y, test_size=0.30, stratify=y, random_state=SEED)
X_val, X_test, y_val, y_test = train_test_split(
    X_tmp, y_tmp, test_size=0.50, stratify=y_tmp, random_state=SEED)
```

```
mu = X_train.mean(axis=0)
sigma = X_train.std(axis=0)
X_train = (X_train - mu) / sigma
X_val = (X_val - mu) / sigma
X_test = (X_test - mu) / sigma

print("Train/Val/Test sizes:", len(X_train), len(X_val), len(X_test))
print("Malignant fraction  :", y_train.mean().round(3),
      y_val.mean().round(3), y_test.mean().round(3))
```

```
Train/Val/Test sizes: 398 85 86
Malignant fraction  : 0.372 0.376 0.372
```

```
# Part (b): Model
def relu(z):
    return np.maximum(0, z)

def sigmoid(z):
    z = np.clip(z, -500, 500)
    return 1.0 / (1.0 + np.exp(-z))

def bce_loss(y_hat, y_true, eps=1e-12):
    y_hat = np.clip(y_hat, eps, 1 - eps)
    return -np.mean(y_true * np.log(y_hat) + (1 - y_true) * np.log(1 - y_hat))


class MLP:
    def __init__(self, layer_sizes=(30, 16, 8, 1), seed=SEED):
        rng = np.random.RandomState(seed)
        self.W, self.b = [], []
        for n_in, n_out in zip(layer_sizes[:-1], layer_sizes[1:]):
            self.W.append(rng.randn(n_in, n_out) * np.sqrt(2.0 / n_in))
            self.b.append(np.zeros((1, n_out)))
        self.cache = {}

    def forward(self, X):
```

```
            a = X
            self.cache = {"a0": X}
            L = len(self.W)
            for l in range(L):
                z = a @ self.W[l] + self.b[l]
                a = sigmoid(z) if l == L - 1 else relu(z)
                self.cache[f"z{l+1}"] = z
                self.cache[f"a{l+1}"] = a
            return a

        def backward(self, y_true, lam=0.0):
            m = y_true.shape[0]
            L = len(self.W)
            dW, db = [None] * L, [None] * L
            dz = (self.cache[f"a{L}"] - y_true) / m
            for l in range(L - 1, -1, -1):
                a_prev = self.cache[f"a{l}"]
                dW[l] = a_prev.T @ dz + 2 * lam * self.W[l]
                db[l] = dz.sum(axis=0, keepdims=True)
                if l > 0:
                    da_prev = dz @ self.W[l].T
                    dz = da_prev * (self.cache[f"z{l}"] > 0)
            return dW, db

        def predict_proba(self, X):
            return self.forward(X)

        def n_params(self):
            return sum(w.size + b.size for w, b in zip(self.W, self.b))

        def weight_norm(self):
            return np.sqrt(sum(np.sum(w ** 2) for w in self.W))

        def get_weights(self):
            return [w.copy() for w in self.W], [b.copy() for b in self.b]

        def set_weights(self, W, b):
            self.W = [w.copy() for w in W]
            self.b = [x.copy() for x in b]


    model = MLP()
    print("Trainable parameters:", model.n_params())
```

```
Trainable parameters: 641
```

```
# Part (c): Gradient Check
def flatten(arrays):
    return np.concatenate([a.ravel() for a in arrays])

def gradient_check(model, X, y, eps=1e-5):
    model.forward(X)
    dW, db = model.backward(y)
    g_analytic = np.concatenate([flatten(dW), flatten(db)])

    params = model.W + model.b
    g_numeric = []
    for p in params:
        flat = p.reshape(-1)
        for i in range(flat.size):
            old = flat[i]
            flat[i] = old + eps
            loss_plus = bce_loss(model.forward(X), y)
```

```
            flat[i] = old - eps
            loss_minus = bce_loss(model.forward(X), y)
            flat[i] = old
            g_numeric.append((loss_plus - loss_minus) / (2 * eps))
    g_numeric = np.array(g_numeric)

    return (np.linalg.norm(g_analytic - g_numeric) /
            (np.linalg.norm(g_analytic) + np.linalg.norm(g_numeric)))

check_model = MLP()
rel_error = gradient_check(check_model, X_train[:8], y_train[:8])
print(f"Gradient check relative error = {rel_error:.2e}")
```

```
Gradient check relative error = 7.23e-11
```

```
# Part (d): Optimizers
class SGD:
    def __init__(self, lr):
        self.lr = lr
    def step(self, params, grads):
        for p, g in zip(params, grads):
            p -= self.lr * g


class Momentum:
    def __init__(self, lr, beta=0.9):
        self.lr, self.beta, self.v = lr, beta, None
    def step(self, params, grads):
        if self.v is None:
            self.v = [np.zeros_like(p) for p in params]
        for p, g, v in zip(params, grads, self.v):
            v *= self.beta
            v += g
            p -= self.lr * v
```

```
class RMSprop:
    def __init__(self, lr, beta=0.9, eps=1e-8):
        self.lr, self.beta, self.eps, self.s = lr, beta, eps, None
    def step(self, params, grads):
        if self.s is None:
            self.s = [np.zeros_like(p) for p in params]
        for p, g, s in zip(params, grads, self.s):
            s *= self.beta
            s += (1 - self.beta) * g ** 2
            p -= self.lr * g / (np.sqrt(s) + self.eps)


class Adam:
    def __init__(self, lr, b1=0.9, b2=0.999, eps=1e-8):
        self.lr, self.b1, self.b2, self.eps = lr, b1, b2, eps
        self.m, self.v, self.t = None, None, 0
    def step(self, params, grads):
        if self.m is None:
            self.m = [np.zeros_like(p) for p in params]
            self.v = [np.zeros_like(p) for p in params]
        self.t += 1
        for p, g, m, v in zip(params, grads, self.m, self.v):
            m *= self.b1; m += (1 - self.b1) * g
            v *= self.b2; v += (1 - self.b2) * g ** 2
```

```
            m_hat = m / (1 - self.b1 ** self.t)
            v_hat = v / (1 - self.b2 ** self.t)
            p -= self.lr * m_hat / (np.sqrt(v_hat) + self.eps)
```

```
def train(model, optimizer, epochs, batch_size=32, lam=0.0,
          patience=None, seed=SEED):
    rng = np.random.RandomState(seed)
    n = X_train.shape[0]
    hist = {"train": [], "val": []}
    best_val, best_weights, wait = np.inf, None, 0

    for epoch in range(epochs):
        idx = rng.permutation(n)
        for start in range(0, n, batch_size):
            batch = idx[start:start + batch_size]
            model.forward(X_train[batch])
            dW, db = model.backward(y_train[batch], lam)
            optimizer.step(model.W + model.b, dW + db)

        tr = bce_loss(model.forward(X_train), y_train)
        va = bce_loss(model.forward(X_val), y_val)
        hist["train"].append(tr)
        hist["val"].append(va)

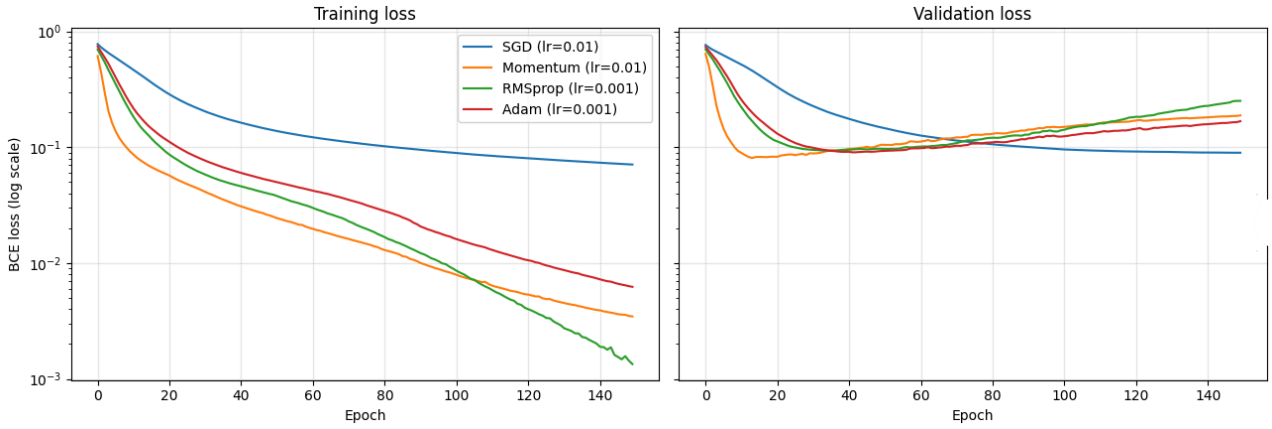
        if patience is not None:
            if va < best_val - 1e-9:
                best_val, best_weights, wait = va, model.get_weights(), 0
            else:
                wait += 1
                if wait >= patience:
                    break
    if patience is not None and best_weights is not None:
        model.set_weights(*best_weights)
    hist["epochs_run"] = len(hist["train"])
    return hist
```

```
optimizers = {
    "SGD (lr=0.01)":      lambda: SGD(0.01),
    "Momentum (lr=0.01)": lambda: Momentum(0.01, 0.9),
    "RMSprop (lr=0.001)": lambda: RMSprop(0.001),
    "Adam (lr=0.001)":    lambda: Adam(0.001),
}
results_d = {}
for name, make_opt in optimizers.items():
    m = MLP()
    results_d[name] = train(m, make_opt(), epochs=150)
    print(f"{name:22s} final train={results_d[name]['train'][-1]:.4f} "
          f"val={results_d[name]['val'][-1]:.4f}")

fig, axes = plt.subplots(1, 2, figsize=(13, 4.5), sharey=True)
for name, h in results_d.items():
    axes[0].plot(h["train"], label=name)
    axes[1].plot(h["val"], label=name)
for ax, title in zip(axes, ["Training loss", "Validation loss"]):
    ax.set_yscale("log"); ax.set_title(title)
    ax.set_xlabel("Epoch"); ax.grid(alpha=0.3)
axes[0].set_ylabel("BCE loss (log scale)")
axes[0].legend()
plt.tight_layout()
```

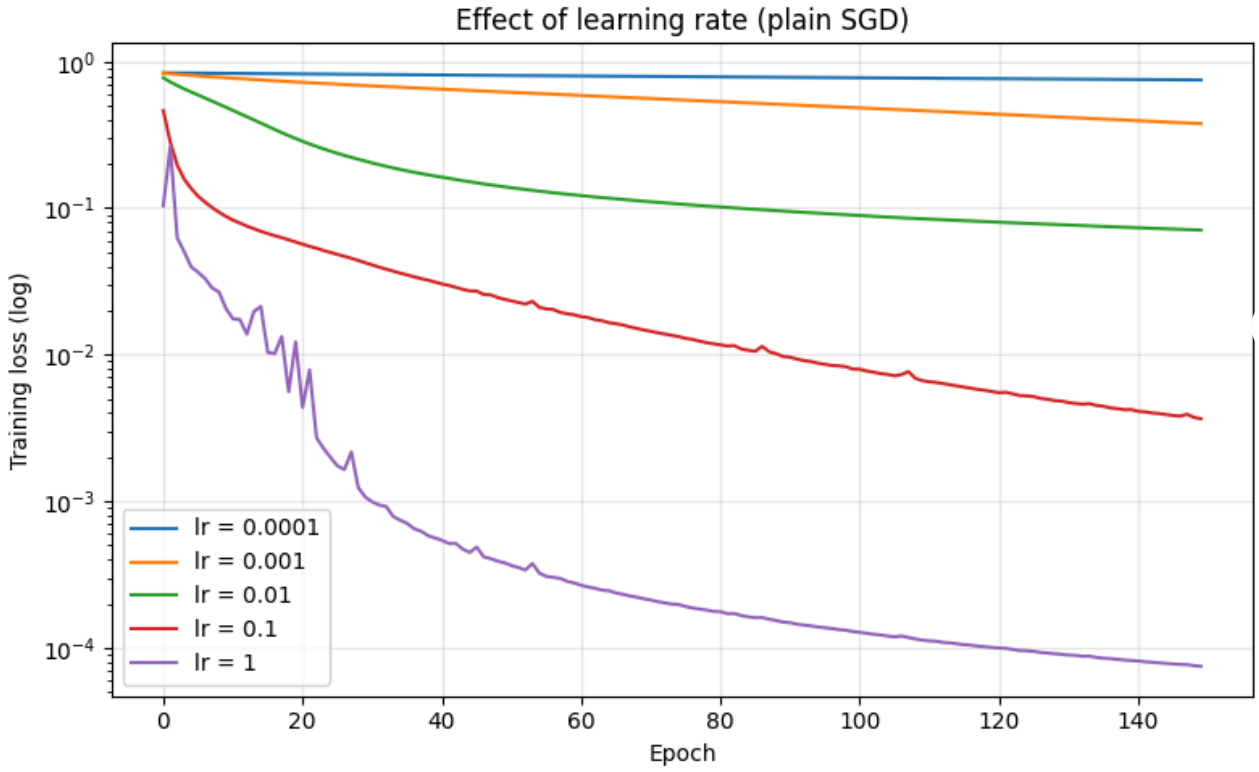
```
plt.savefig("q5d_optimizers.png", dpi=150)
plt.show()
```

```
SGD (lr=0.01)          final train=0.0708 val=0.0895
Momentum (lr=0.01)     final train=0.0035 val=0.1881
RMSprop (lr=0.001)     final train=0.0013 val=0.2509
Adam (lr=0.001)        final train=0.0062 val=0.1675
```




```
# Part (e): Learning Rate
lrs = [1e-4, 1e-3, 1e-2, 1e-1, 1.0]
plt.figure(figsize=(8, 5))
for lr in lrs:
    m = MLP()
    with np.errstate(all="ignore"):
        h = train(m, SGD(lr), epochs=150)
    plt.plot(h["train"], label=f"lr = {lr:g}")
    print(f"SGD lr={lr:<7g} final train loss = {h['train'][-1]:.4f}")
plt.yscale("log"); plt.xlabel("Epoch"); plt.ylabel("Training loss (log)")
plt.title("Effect of learning rate (plain SGD)")
plt.legend(); plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("q5e_learning_rate.png", dpi=150)
plt.show()
```

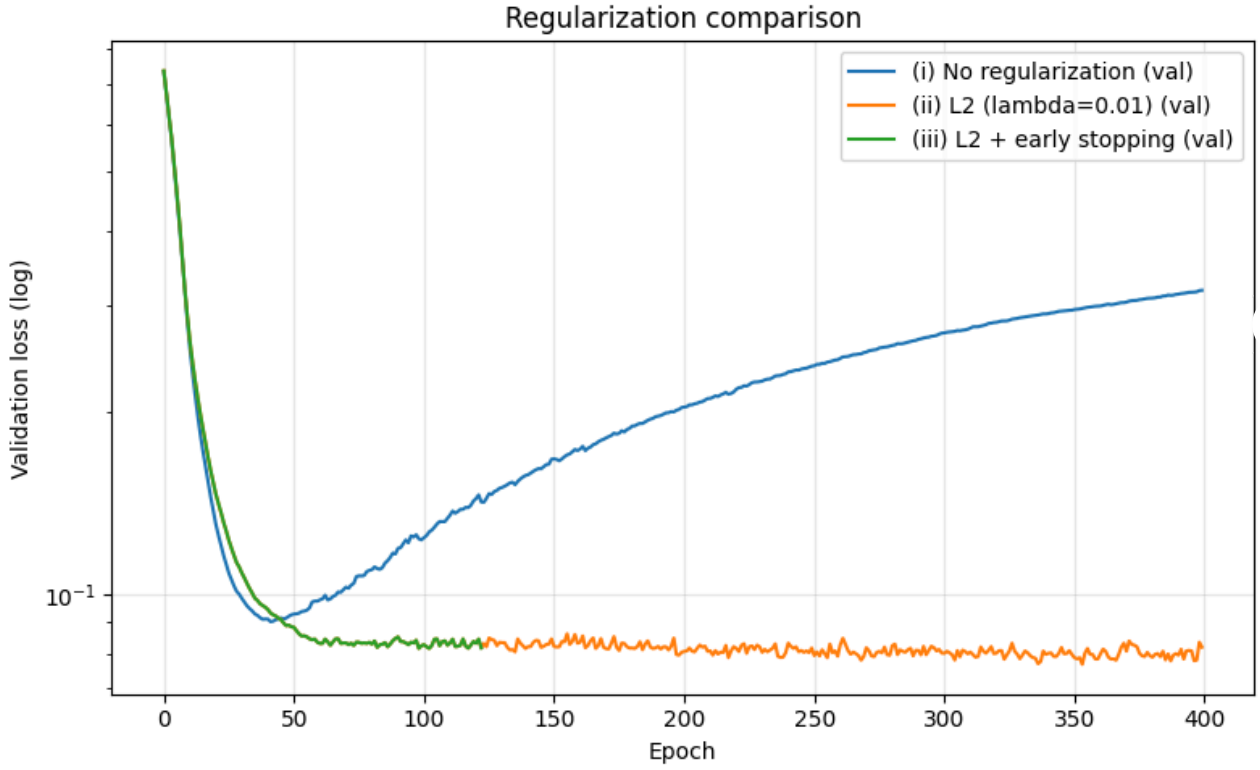
```
SGD lr=0.0001  final train loss = 0.7494
SGD lr=0.001   final train loss = 0.3771
SGD lr=0.01    final train loss = 0.0708
SGD lr=0.1     final train loss = 0.0037
SGD lr=1       final train loss = 0.0001
```




```
# Part (f): Regularization
configs = {
    "(i) No regularization":     dict(lam=0.0,  patience=None),
    "(ii) L2 (lambda=0.01)":     dict(lam=0.01, patience=None),
    "(iii) L2 + early stopping": dict(lam=0.01, patience=20),
}
models_f, hist_f = {}, {}
print(f"\n{'Setting':28s}{'Epochs':>8s}{'Train loss':>12s}{'Val loss':>10s}{'||W||_F':>10s}")
for name, cfg in configs.items():
    m = MLP()
    h = train(m, Adam(0.001), epochs=400, **cfg)
    models_f[name], hist_f[name] = m, h
    tr = bce_loss(m.forward(X_train), y_train)
    va = bce_loss(m.forward(X_val), y_val)
    print(f"{name:28s}{h['epochs_run']:>8d}{tr:>12.4f}{va:>10.4f}{m.weight_norm():>10.4f}")

plt.figure(figsize=(8, 5))
for name, h in hist_f.items():
    plt.plot(h["val"], label=name + " (val)")
plt.yscale("log"); plt.xlabel("Epoch"); plt.ylabel("Validation loss (log)")
plt.title("Regularization comparison"); plt.legend(); plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("q5f_regularization.png", dpi=150)
plt.show()
```

```
Setting                      Epochs  Train loss  Val loss   ||W||_F
(i) No regularization           400      0.0002    0.3187    9.9044
(ii) L2 (lambda=0.01)           400      0.0582    0.0819    2.6791
(iii) L2 + early stopping       123      0.0577    0.0815    3.0334
```




```
# Part (g): Evaluation
final_model = models_f["(iii) L2 + early stopping"]

def evaluate(model, X, y, threshold=0.5):
    y_pred = (model.predict_proba(X) >= threshold).astype(int)
    TP = int(np.sum((y_pred == 1) & (y == 1)))
    TN = int(np.sum((y_pred == 0) & (y == 0)))
    FP = int(np.sum((y_pred == 1) & (y == 0)))
    FN = int(np.sum((y_pred == 0) & (y == 1)))
    acc = (TP + TN) / len(y)
    prec = TP / (TP + FP) if (TP + FP) else 0.0
    rec = TP / (TP + FN) if (TP + FN) else 0.0
    f1 = 2 * prec * rec / (prec + rec) if (prec + rec) else 0.0
    return acc, prec, rec, f1, np.array([[TN, FP], [FN, TP]])

acc, prec, rec, f1, cm = evaluate(final_model, X_test, y_test)
print("\nTEST RESULTS (threshold 0.5)")
print(f"Accuracy : {acc:.4f}")
print(f"Precision: {prec:.4f}")
print(f"Recall   : {rec:.4f}")
print(f"F1-score : {f1:.4f}")
print("Confusion matrix (rows = actual, cols = predicted)")
print("               Pred Benign  Pred Malignant")
print(f"Actual Benign      {cm[0,0]:5d}        {cm[0,1]:5d}")
print(f"Actual Malignant   {cm[1,0]:5d}        {cm[1,1]:5d}")

print("\nThreshold effect:")
print("threshold  recall  precision  FN  FP")
for t in [0.1, 0.3, 0.5, 0.7, 0.9]:
    _, p, r, _, c = evaluate(final_model, X_test, y_test, threshold=t)
    print(f"   {t:.1f}     {r:.3f}   {p:.3f}     {c[1,0]:2d}  {c[0,1]:2d}")
```

```
TEST RESULTS (threshold 0.5)
Accuracy : 0.9767
Precision: 1.0000
Recall   : 0.9375
F1-score : 0.9677
Confusion matrix (rows = actual, cols = predicted)
               Pred Benign  Pred Malignant
Actual Benign         54            0
Actual Malignant       2           30
```